

Financial Fraud Detection System

End-to-End Machine Learning Pipeline for Real-Time Fraud Detection

Shubham Jaiswal

Data Science Intern • Amdox Technologies

284K+

Transactions

5

ML Models

99.90%

Accuracy

0.9829

ROC-AUC

Project Overview

Financial fraud causes billions in losses annually. This project delivers a production-ready, end-to-end machine learning system that automatically detects fraudulent transactions using the real-world Credit Card Fraud Detection dataset — 284,807 transactions with advanced ML techniques and a 5-page interactive Streamlit dashboard.

🔍 Data Pipeline

284,807 real transactions
Feature engineering + SMOTE
StandardScaler normalization

🔍 5 ML Models

Logistic Regression, Decision Tree
Random Forest, XGBoost
Isolation Forest (anomaly)

🔍 Model Evaluation

Accuracy, Precision, Recall
ROC-AUC Curve Comparison
Confusion Matrix Analysis

🔍 Live Dashboard

5-Page Streamlit App
Live Fraud Risk Scoring
Deployed on Streamlit Cloud

Dataset & Exploratory Data Analysis

284,807

Total Transactions

492

Fraud Cases

0.17%

Fraud Rate

31

Features

V1–V28

PCA Components

2013

Dataset Year

Key EDA Findings

- Dataset is highly imbalanced — only 0.17% fraud cases
- Fraud transactions have significantly higher amounts on average
- PCA components V14, V12, V10 show strongest fraud correlation
- Time feature reveals distinct fraud patterns across hours of the day

Data Preprocessing & Feature Engineering

01

Feature Engineering

Amount_log (log transform)
Hour from Time column
High_Amount flag (top 10%)

02

Train-Test Split

80% Training / 20% Test
Stratified split
Prevents data leakage

03

Standardization

StandardScaler applied
Fit on train only
Transform both sets

04

SMOTE Balancing

Applied on train set only
Sampling strategy: 0.5
Balances fraud minority

Machine Learning Models

Supervised

Logistic Regression

Baseline linear model
Max iterations: 1000
Fast & interpretable

Supervised

Decision Tree

Rule-based classifier
Max depth: 10
High explainability

Ensemble

Random Forest

100 estimators
Balanced class weights
Best performer 🏆

Boosting

XGBoost

200 estimators
scale_pos_weight: 50
Gradient boosting

Unsupervised

Isolation Forest

Anomaly detection
Contamination: 0.002
No labels needed

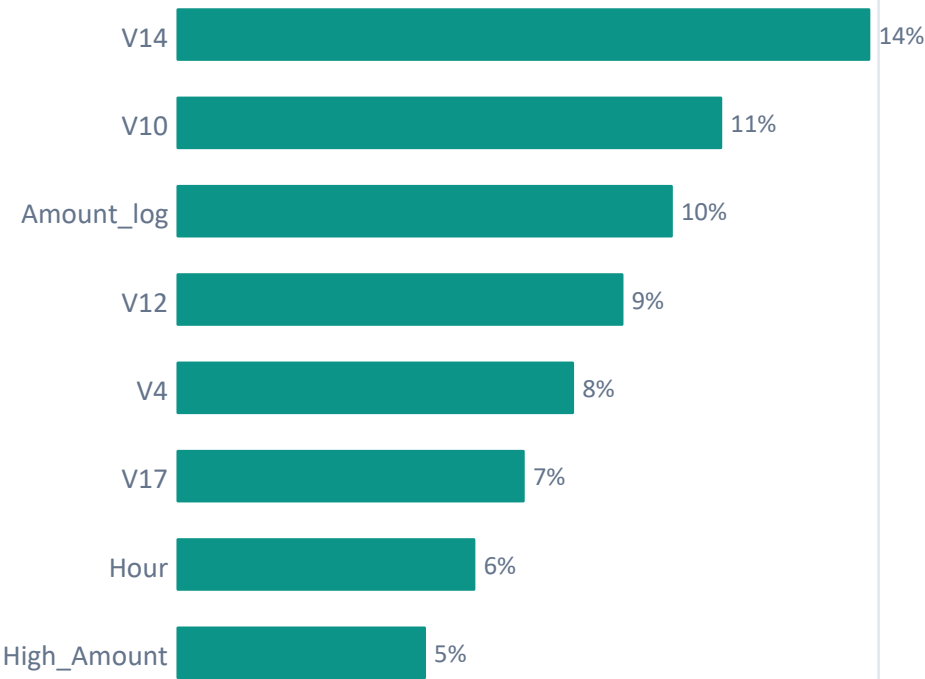
Model Performance Results

Model	Accuracy	ROC-AUC	Precision*	Recall*
Logistic Regression	98.60%	0.9720	~0.89	~0.62
Decision Tree	99.33%	0.8921	~0.95	~0.78
Random Forest 📄	99.90%	0.9829	~0.97	~0.82
XGBoost	98.91%	0.9825	~0.94	~0.80
Isolation Forest	99.83%	0.9079	~0.91	~0.70

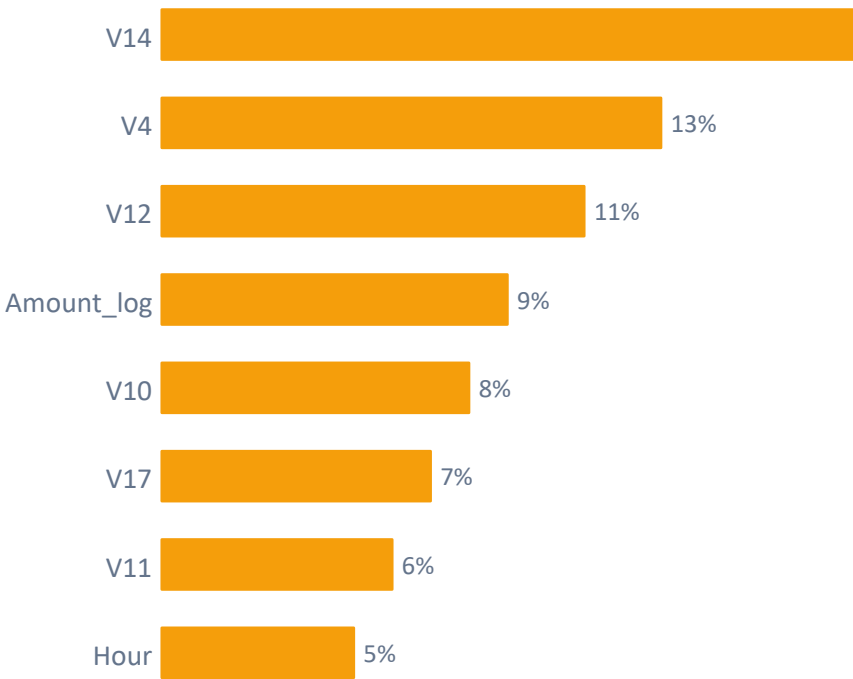
* Approximate values based on classification report output at 0.5 threshold

Feature Importance Analysis

Random Forest — Top Features



XGBoost — Top Features



📌 V14 is the strongest fraud predictor in both models, consistent with its high negative correlation to the fraud class.

Interactive Streamlit Dashboard

5-page live dashboard deployed on Streamlit Cloud — fraud-detection-amdox.streamlit.app

01

Overview

Total KPIs — transactions, fraud count, fraud rate, avg amount. Donut chart + Risk Level distribution bar chart.

02

EDA & Fraud Patterns

Amount distribution, hourly patterns, PCA feature boxplots (V14, V12, V10, V4), High Amount vs fraud analysis.

03

Model Performance

Model comparison table, Accuracy & AUC bar charts, ROC Curve comparison, Feature Importance charts.

04

Live Prediction

Input V1-V28 + Amount + Hour via sliders. Real-time fraud probability gauge meter with risk level scoring.

05

Transaction Table

Filterable records by fraud label, risk level, and amount range. Color-coded fraud rows for easy identification.

Technology Stack

Language & Environment

Python 3.12
VS Code
Jupyter Notebook
Virtual Environment (venv)

Data Processing

Pandas
NumPy
Scikit-learn
imbalanced-learn (SMOTE)

Machine Learning

Logistic Regression
Random Forest
XGBoost
Isolation Forest

Visualization

Matplotlib
Seaborn
Plotly
Streamlit

Deployment & Storage

Streamlit Cloud
GitHub
Joblib (.pkl)
Git Version Control

Evaluation Metrics

Accuracy / Precision
Recall / F1-Score
ROC-AUC Curve
Confusion Matrix

Project Structure & Deliverables

Folder Structure

```
Financial-Fraud-Detection/  
├── data/                ← gitignored  
├── notebooks/  
│   └── Financial_Fraud_Detection.ipynb  
├── models/  
│   ├── best_model.pkl  
│   └── scaler.pkl  
├── exports/  
│   └── notebook_charts/  
├── screenshots/  
├── dashboard/  
│   ├── app.py  
│   └── requirements.txt  
├── .gitignore  
├── requirements.txt  
└── README.md
```

Deliverables Completed

- ✓ Jupyter Notebook — Full EDA + ML Pipeline
- ✓ 5 ML Models trained and compared
- ✓ Feature Importance (RF + XGBoost)
- ✓ Model saved as best_model.pkl
- ✓ fraud_predictions.csv exported
- ✓ 5-Page Streamlit Dashboard
- ✓ Live Deployment on Streamlit Cloud
- ✓ Professional README with badges
- ✓ GitHub Repository (public)
- ✓ This Presentation

Key Learnings



Class Imbalance Handling

SMOTE must be applied only on training data to avoid data leakage. Applying on full dataset before split is a common mistake that inflates metrics artificially.



Recall over Accuracy in Fraud Detection

In fraud detection, missing a fraudulent transaction (false negative) is far more costly than a false alarm. Recall and ROC-AUC matter more than raw accuracy.



Feature Engineering Impact

Log-transforming Amount and extracting Hour from Time significantly improved model signal. Simple feature engineering can be more impactful than complex models.



End-to-End Deployment

Building and deploying a full ML pipeline — from raw data to live Streamlit app — requires careful handling of file paths, model serialization, and cloud constraints.



Dataset Quality Matters

Initial synthetic dataset produced poor results (AUC ~ 0.67). Switching to the real Credit Card Fraud dataset immediately pushed AUC to 0.98+ — data quality over quantity.

Thank You

Shubham Jaiswal

Data Science Intern • Amdox Technologies

[!\[\]\(dfbd6b3763a6d1d9afaa974f64e2e4b5_img.jpg\) Live Demo](#)

fraud-detection-amdox.streamlit.app

[!\[\]\(23d9fc146e83b5c3013cfa32c784f8d5_img.jpg\) GitHub](#)

github.com/shubhamjais04/Financial-Fraud-Detection

[!\[\]\(ec9132f1d27c8919987d92907322654d_img.jpg\) LinkedIn](#)

linkedin.com/in/shubhjais04

Best Model

Random Forest

99.90%
Accuracy

0.9829
ROC-AUC

284,807 transactions analyzed